20. ArrayList Working (Internal Working) — Quick Notes

Internal Structure

ArrayList Internally Uses

Normal Array
ArrayList is basically: Array + Dynamic Resizing Logic
Examples in other languages:

- C++ → Vector
- Python → List

Dynamic Resizing Concept

Suppose initial capacity: 10
Elements added: [1,2,3,4,5,6,7,8,9,10]
Array becomes full.
When 11th element comes:

Step 1: Create bigger array. Size 10 → Size 20

Step 2: Copy old elements. -> 1..10 to new array.

Step 3: Insert new element. -> 11

Step 4: Delete old array.

Growth Pattern

For explanation: 10 → 20 → 40 → 80 → 160

Actual Java (Modern JDK)

Usually: newCapacity = oldCapacity + (oldCapacity / 2)

Example: 10 → 15 → 22 → 33 → 49

Implementation may vary between JDK versions.

Why ArrayList is Fast?

Because data is stored in: Contiguous Memory Locations
Example: |10|20|30|40|50|

Advantages:

1. Cache Friendly

CPU can prefetch nearby elements.

2. Random Access

list.get(i);
Time Complexity: O(1)
Unlike:

- LinkedList
- TreeSet
- TreeMap

Example

```
ArrayList<Integer> list = new ArrayList<>();
```

```
for(int i=1;i<=100;i++)
  list.add(i);
```

Internally:
Capacity 10
↓
Capacity 20
↓
Capacity 40
↓
Capacity 80
↓
Capacity 160

Size: 100
Capacity:160

Size vs Capacity

Property	Meaning
Size	Actual elements
Capacity	Internal storage

Example:

ArrayList<Integer> list = new ArrayList<>(100);
Initially: Size = 0, Capacity = 100

Important

Java provides: list.size()
But: ❌ No public **list.capacity()** method.

Time Complexity of add()

Normal Case

list.add(x);
Just place at end.
O(1)

Resize Case

When array becomes full:

Create new array
+
Copy elements
+
Insert new element

Time: O(n)
Example: Insert 11th element after capacity 10 is full.

Worst Case vs Amortized

Operation	Complexity
add() Worst Case	O(n)
add() Amortized	O(1)

Interview Point

Most insertions: O(1)
Only occasional resizing: O(n)
Average over many operations: Amortized O(1)

Why Amortized O(1)?

Assume: Capacity = n

First: n insertions
Cost: O(1) each.

Next insertion: Resize + Copy
Cost: O(n) But this expensive operation happens rarely.

Average cost over all insertions: O(1)
This is called: Amortized Analysis

Avoid Resizing Cost

Method 1
Specify Capacity

```
ArrayList<Integer> list = new ArrayList<>(100);
```

Method 2
ensureCapacity()

```
ArrayList<Integer> list = new ArrayList<>();

list.ensureCapacity(100);
```

Guarantees: Capacity >= 100
No resizing until 100 elements.

---

ensureCapacity()

Example:

```
ArrayList<Integer> list = new ArrayList<>();

list.ensureCapacity(1000);
```

Benefits:

- Fewer reallocations
- Better performance
- Useful when expected size known later

Array vs ArrayList

Feature	Array	ArrayList
Size	Fixed	Dynamic
Random Access	O(1)	O(1)
Resize	Manual	Automatic
Memory Overhead	Less	More
Library Methods	Few	Rich API

Important Quick Points

Topic	Point
Internal DS	Array
Dynamic Resize	Yes
Random Access	O(1)
Cache Friendly	Yes
Worst Case add()	O(n)
Amortized add()	O(1)
Capacity Growth	~1.5x (modern JDK)
Size	Actual elements
Capacity	Internal storage
ensureCapacity()	Pre-allocate memory
Best Use Case	Frequent reads + append operations